

CHANDIGARH UNIVERSITY

DEPARTMENT: UIC | ACADEMIC SESSION 2026-2027

MINI PROJECT REPORT | BCA

CPU Scheduling Algorithm Simulator

A C++ Mini Project for Object Oriented Programming

FCFS | SJF | PRIORITY | ROUND ROBIN

STUDENT / PROJECT DETAILS

Student Name	**KARTTAVYA SANGRAJ**
UID / Registration	**25BCA110097**
Course & Programme	**BCA (General)**
Subject	**Object Oriented Programming (25CAH-201)**
Department	**UIC**
University / College	**Chandigarh University**
Submitted To	**Mrs. Rashmi**
Academic Session	**2026-2027**

Project domain: Operating Systems concepts implemented using C++

Certificate & Declaration

Academic authenticity of the submitted work

CERTIFICATE

This is to certify that the mini project entitled **CPU Scheduling Algorithm Simulator** has been prepared and submitted as academic work for **Object Oriented Programming (25CAH-201)**. It demonstrates CPU scheduling concepts through a menu-driven C++ application.

Faculty Signature (Mrs. Rashmi): _____ Date: _____

STUDENT DECLARATION

I declare that this mini project is my academic work, prepared for learning and demonstration purposes. Concepts and definitions from reference material are acknowledged in the references section.

Student Signature: _____ Karttavya Sangraj | 25BCA110097

Contents

01	Abstract and Objectives	3
02	Problem Statement and Requirements	4
03	Conceptual Background	5
04	System Design and Flow	6
05	Implementation and OOP Discussion	7
06	Testing and Sample Output	8
07	Results (Measured from the Sample Run)	9
08	Limitations and Future Scope	11
09	Conclusion, References and Execution	12

01 Abstract & Objectives

What the simulator does and what it teaches

Abstract

The CPU Scheduling Algorithm Simulator is a console-based C++ application demonstrating how an operating system selects processes for CPU execution. It accepts process arrival time, burst time and priority, then simulates **FCFS**, **non-preemptive SJF**, **non-preemptive Priority** and **Round Robin**. It displays a Gantt chart, completion time, turnaround time, waiting time, response time and average metrics. The project connects Operating Systems theory with programming practice using structures, vectors, functions, loops, conditional statements and a polymorphic class hierarchy.

4	1-20	5	1
SCHEDULING ALGORITHMS	PROCESSES SUPPORTED	METRICS REPORTED	CPU CORE MODELLED

Objectives

Understand scheduling
Understand CPU scheduling in a multitasking operating system.

Implement four algorithms
Implement four commonly taught scheduling algorithms.

Calculate key metrics
Calculate waiting, turnaround and response times.

Visualise execution
Represent execution order using a Gantt chart.

Robust, modular code
Apply input validation and modular programming.

Apply OOP
Use inheritance and runtime polymorphism through a strategy hierarchy.

Scope

The simulator is intended for classroom learning. It assumes integer time units, one CPU, no context-switch overhead, and non-preemptive SJF and Priority. Round Robin is preemptive.

02 Problem Statement & Requirements

Why the tool is needed and what it must do

Problem Statement

Manually comparing scheduling algorithms for different arrival and burst times can be time-consuming and error-prone. This project automates the calculations and makes process execution order visible through a Gantt chart.

Functional Requirements

FR1 Process input
Read between 1 and 20 processes.

FR2 Input rules
Accept non-negative arrival time and priority, and positive burst time.

FR3 Algorithm choice
Allow one algorithm or all algorithms to be executed.

FR4 Round Robin
Ask for a positive time quantum for Round Robin.

FR5 Output
Display a Gantt chart and a process-wise result table.

FR6 Reliability
Re-prompt on invalid input instead of crashing.

Hardware and Software

HARDWARE	Ordinary laptop or desktop computer.
OPERATING SYSTEM	Windows, Linux or macOS.
COMPILER	GCC / MinGW or another modern C++ compiler (the project is compiled with C++17).
TOOLS	Terminal or an IDE such as Code::Blocks, VS Code or Visual Studio.

03 Conceptual Background

Scheduling theory behind the implementation

CPU Scheduling

CPU scheduling is the operating-system activity of choosing a process from the ready queue and allocating the CPU to it. A scheduling policy affects responsiveness, throughput and time spent waiting.

Algorithms Implemented

FCFS - First Come First Served

Executes processes in order of arrival. A long process can delay shorter ones behind it (the convoy effect).

SJF - Shortest Job First

Non-preemptive. Selects the available process with the smallest burst time and runs it to completion.

Priority Scheduling

Non-preemptive. Selects the available process with the highest priority. A smaller numeric value means higher priority here.

Round Robin

Each ready process receives CPU time up to a fixed quantum; unfinished work returns to the back of the queue.

Quick Comparison

Algorithm	Selection rule	Preemptive	Typical trade-off
FCFS	Earliest arrival	No	Simple, but long jobs delay short ones
SJF	Smallest burst time	No	Low average waiting; long jobs may wait
Priority	Smallest priority number	No	Honours importance; low priority may starve
Round Robin	FIFO queue with fixed quantum	Yes	Fair, good response; more switching

Performance Formulae

TAT = CT - AT
TURNAROUND TIME

WT = TAT - BT
WAITING TIME

RT = Start - AT
RESPONSE TIME (first CPU start)

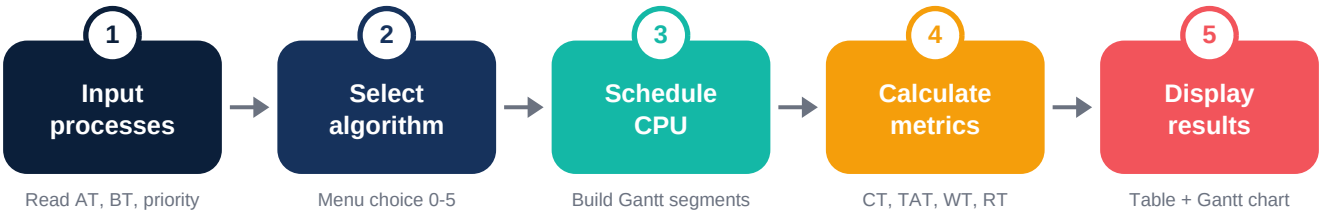
Average metrics equal the sum of the process metrics divided by the number of processes.

04 System Design & Flow

From raw input to results on screen

Program Workflow

The program collects process data, initializes remaining burst time and asks the user to select a scheduling policy. The selected algorithm builds a sequence of CPU execution segments, including idle segments when no process is ready. It then calculates metrics and prints a result table and Gantt chart.



Algorithm Logic

FCFS

Execute in arrival order. Insert an IDLE segment if the CPU is free before the next arrival.

SJF

At each decision point, choose the available process with the shortest burst; ties go to earlier arrival.

Priority

Choose the available process with the smallest priority number; ties go to earlier arrival.

Round Robin

Allocate a fixed time quantum in FIFO order, track remaining burst time and re-queue unfinished work.

Data Flow of a Single Segment

Every algorithm appends **Segment** records (process id, start, end) to a chart vector. Process id **0** represents an idle CPU. After scheduling, `finishMetrics()` derives turnaround and waiting time, and `printResults()` renders the table, averages and Gantt chart from the same data, so the displayed chart always agrees with the computed metrics.

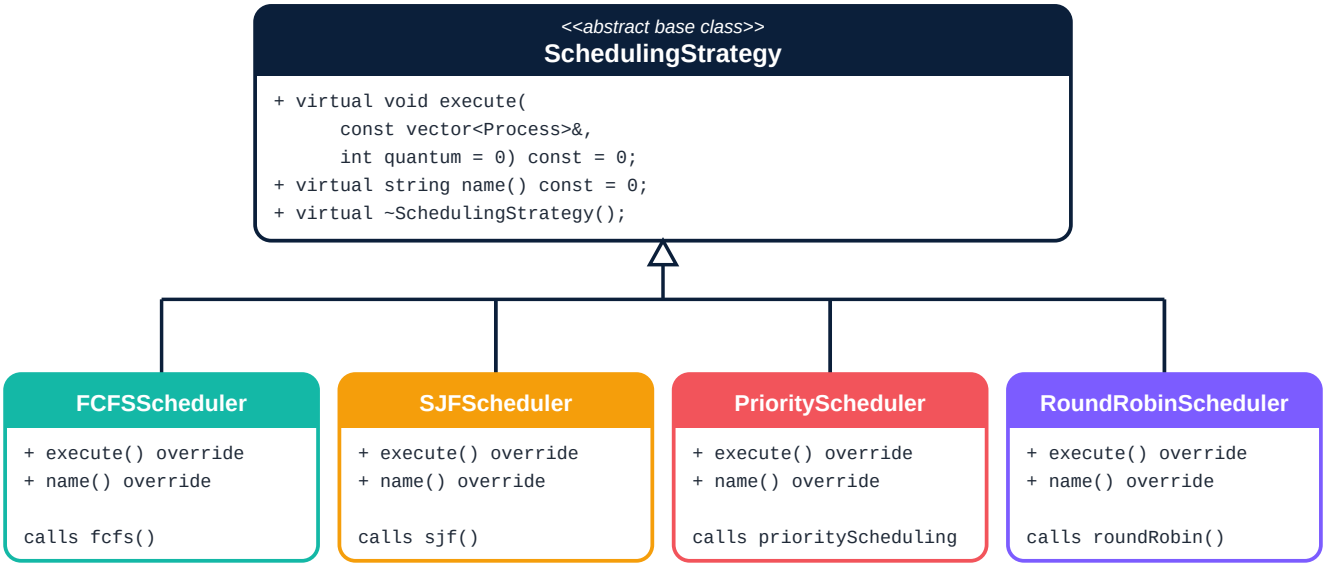
05 Implementation & OOP

Data representation, modular functions and class design

Language and Data Representation

The project is written in standard C++. A **Process** structure stores process ID, arrival time, burst time, priority and calculated metrics. A **Segment** structure stores one Gantt-chart interval; process ID 0 represents CPU idle time. Vectors are used because the number of processes is entered at runtime.

OOP Design: Strategy Hierarchy



Inheritance (is-a): each derived class overrides the pure virtual methods of the base class.

```
const SchedulingStrategy* strategy = &scheduler;
strategy->execute(p); // runtime polymorphism via base-class pointer
```

Modular Functions

`readProcesses()`	Collects and validates process information.
`fcfs()`	Schedules processes in arrival order.
`sjf()`	Selects the shortest available burst time.
`priorityScheduling()`	Selects the highest-priority available process.
`roundRobin()`	Uses a ready queue and a user-defined time quantum.
`finishMetrics()`, `printResults()`, `printGantt()`	Calculate and display results.

Design assumptions: SJF and Priority are non-preemptive. Ties are resolved by earlier arrival. Context-switch cost, I/O blocking, multiple CPU cores and process aging are outside the current scope.
OOP learning outcomes: an abstract `SchedulingStrategy` base class and four derived classes override the virtual `execute()` method; the main menu invokes every algorithm through a base-class pointer. **Process** and **Segment** structures group related data.

06 Testing & Sample Output

Verification using the captured sample run

Testing Strategy

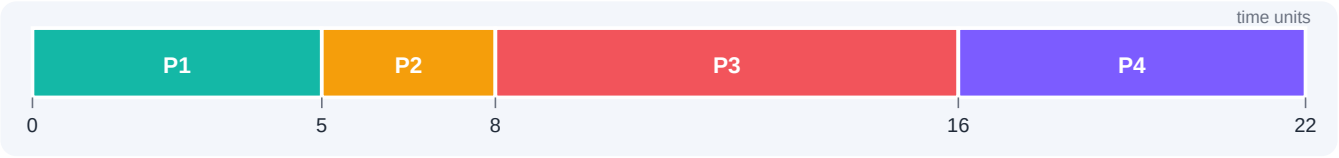
The program is compiled using g++ with C++17. Test cases should include processes arriving at time zero, staggered arrivals, CPU idle gaps, equal burst times, equal priorities and different positive Round Robin quantum values. Invalid process counts, non-positive burst times and invalid quantum values are re-prompted.

Sample Input Set

Process	Arrival (AT)	Burst (BT)	Priority
P1	0	5	2
P2	1	3	1
P3	2	8	3
P4	3	6	2

A captured execution of this input is included in Sample_Run.txt.

FCFS Gantt Chart for the Sample Input



Expected Checks (verified on the sample run)

Check	Evidence from the sample run	
TAT must equal CT - AT for every process.	FCFS P3: 16 - 2 = 14; RR P4: 20 - 3 = 17.	PASS
WT must equal TAT - BT and cannot be negative.	SJF P4: 11 - 6 = 5; all waiting times are 0 or more.	PASS
RT must equal first start time - arrival time.	Round Robin P4: first start 8, arrival 3, RT = 5.	PASS
Total CPU execution must equal the sum of all burst times.	5 + 3 + 8 + 6 = 22 = final Gantt time in every algorithm.	PASS

07 Results

Measured values from Sample_Run.txt (4 processes, quantum = 2)

FIRST COME FIRST SERVED (FCFS)

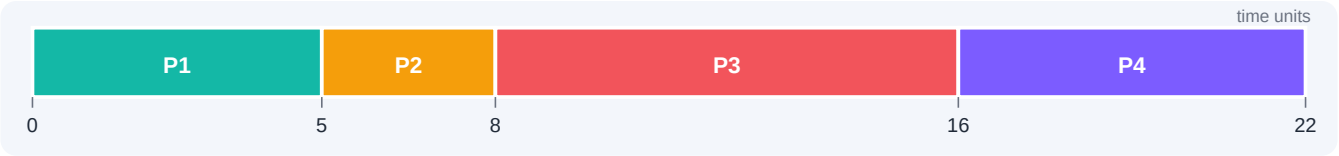
Non-preemptive | arrival order

Process	AT	BT	Priority	CT	TAT	WT	RT
P1	0	5	2	5	5	0	0
P2	1	3	1	8	7	4	4
P3	2	8	3	16	14	6	6
P4	3	6	2	22	19	13	13

5.75 AVERAGE WAITING TIME

11.25 AVERAGE TURNAROUND TIME

5.75 AVERAGE RESPONSE TIME



SJF and PRIORITY (NON-PREEMPTIVE)

Identical schedule for this input

Process	AT	BT	Priority	CT	TAT	WT	RT
P1	0	5	2	5	5	0	0
P2	1	3	1	8	7	4	4
P3	2	8	3	22	20	12	12
P4	3	6	2	14	11	5	5

5.25 AVERAGE WAITING TIME

10.75 AVERAGE TURNAROUND TIME

5.25 AVERAGE RESPONSE TIME



For this workload, SJF and Priority both choose P4 before P3 at time 8 (burst 6 vs 8, priority 2 vs 3), so they produce the same Gantt chart and averages.

07 Results (continued)

Round Robin and algorithm comparison

ROUND ROBIN (QUANTUM = 2)

Preemptive | FIFO ready queue

Process	AT	BT	Priority	CT	TAT	WT	RT
P1	0	5	2	14	14	9	0
P2	1	3	1	11	10	7	1
P3	2	8	3	22	20	12	2
P4	3	6	2	20	17	11	5

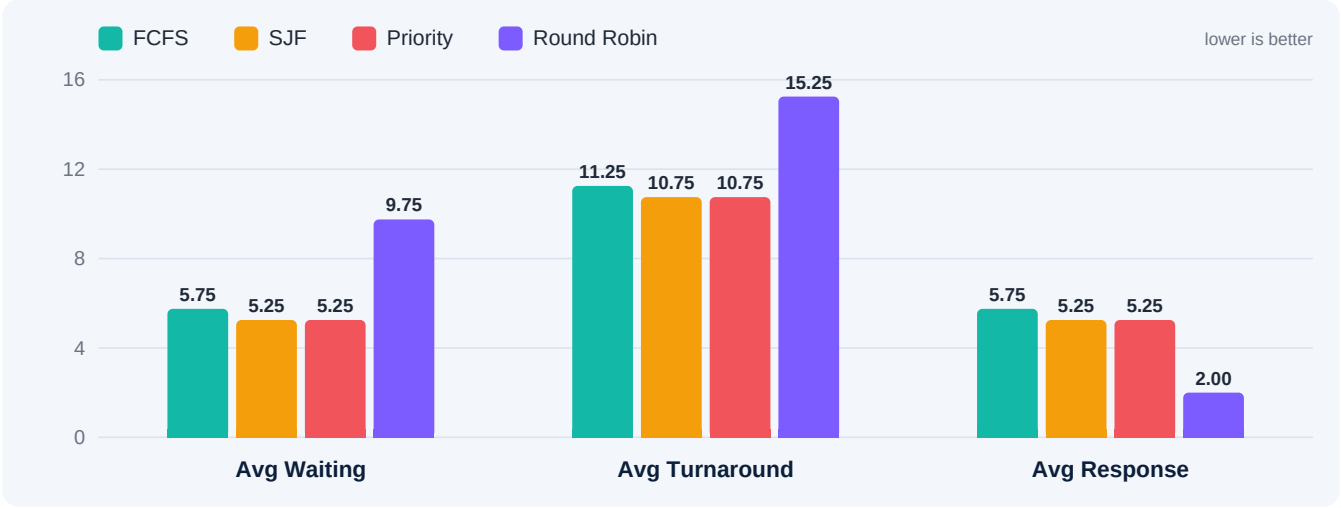
9.75 AVERAGE WAITING TIME

15.25 AVERAGE TURNAROUND TIME

2.00 AVERAGE RESPONSE TIME



Average Metrics Compared



What the Data Shows

- **SJF / Priority** gave the lowest average waiting (5.25) and turnaround (10.75) time on this input.
- **Round Robin** gave the best average response time (2.00): every process got the CPU within 5 time units.
- **Round Robin** also had the highest average waiting time (9.75) because processes are repeatedly interrupted.
- **FCFS** shows the convoy effect: P4 waited 13 units behind the long job P3.

Actual averages depend on the entered workload; no algorithm is best for every possible workload.

08 Limitations & Future Scope

Honest boundaries and the road ahead

Limitations

Non-preemptive only
SJF and Priority are non-preemptive only.

Simplified hardware model
No context-switch overhead, I/O burst or multi-core simulation.

No aging analysis
Priority aging and starvation analysis are not included.

Terminal interface
The interface is terminal-based.

Future Enhancements

Preemptive variants
Add preemptive SJF (SRTF) and preemptive Priority.

GUI and reports
Add a graphical interface and exportable reports.

Live charts
Generate comparison charts from actual simulation data.

Realistic modelling
Model context-switch overhead, multiple CPU cores and priority aging.

Development Roadmap



09 Conclusion, References & Execution

Wrapping up and how to run the project

Conclusion

The CPU Scheduling Algorithm Simulator is a functional C++ mini project connecting Operating Systems theory with programming practice. It implements FCFS, SJF, Priority and Round Robin, handles arrival times and idle CPU periods, validates important inputs and calculates standard scheduling metrics.

How to Compile and Run

1. Install a C++ compiler such as GCC.
2. Open a terminal in the project folder.
3. Compile the program (command below).
4. Run ./cpu_scheduler (Windows: cpu_scheduler.exe).
5. Enter process details and select an algorithm.

```
$ g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o cpu_scheduler
$ ./cpu_scheduler
```

References

- Silberschatz, Galvin and Gagne, *Operating System Concepts*, CPU Scheduling chapter.
- William Stallings, *Operating Systems: Internals and Design Principles*.
- C++ standard library reference: cppreference.com.
- Course notes and classroom material on Operating Systems and Object Oriented Programming.

End of Report